

Universidad de Oriente	
Ciencia de datos	Clase 04 - Ejemplo 2 - EDA

```
In [9]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
```

Una empresa dedicada a la producción y comercialización de vinos desea analizar las características fisicoquímicas de sus productos con el propósito de comprender cuáles de ellas podrían estar relacionadas con la calidad del vino.

Cada observación representa una muestra de vino, para la cual se registraron diferentes propiedades fisicoquímicas y una valoración de calidad.

Análisis descriptivo de los datos

```
In [12]: calidad_vino = pd.read_csv("WineQT.csv")
```

```
In [14]: calidad_vino
```

```
Out[14]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulph
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	
...	...	...	...	...	...	...	...	...	...	...
1138	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	
1139	6.8	0.620	0.08	1.9	0.068	28.0	38.0	0.99651	3.42	
1140	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	
1141	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	
1142	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	

1143 rows × 13 columns



```
In [16]: calidad_vino.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          1143 non-null   float64
1   volatile acidity       1143 non-null   float64
2   citric acid            1143 non-null   float64
3   residual sugar         1143 non-null   float64
4   chlorides              1143 non-null   float64
5   free sulfur dioxide    1143 non-null   float64
6   total sulfur dioxide   1143 non-null   float64
7   density                1143 non-null   float64
8   pH                    1143 non-null   float64
9   sulphates              1143 non-null   float64
10  alcohol                1143 non-null   float64
11  quality                1143 non-null   int64
12  Id                     1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

In [18]: `calidad_vino.describe().T`

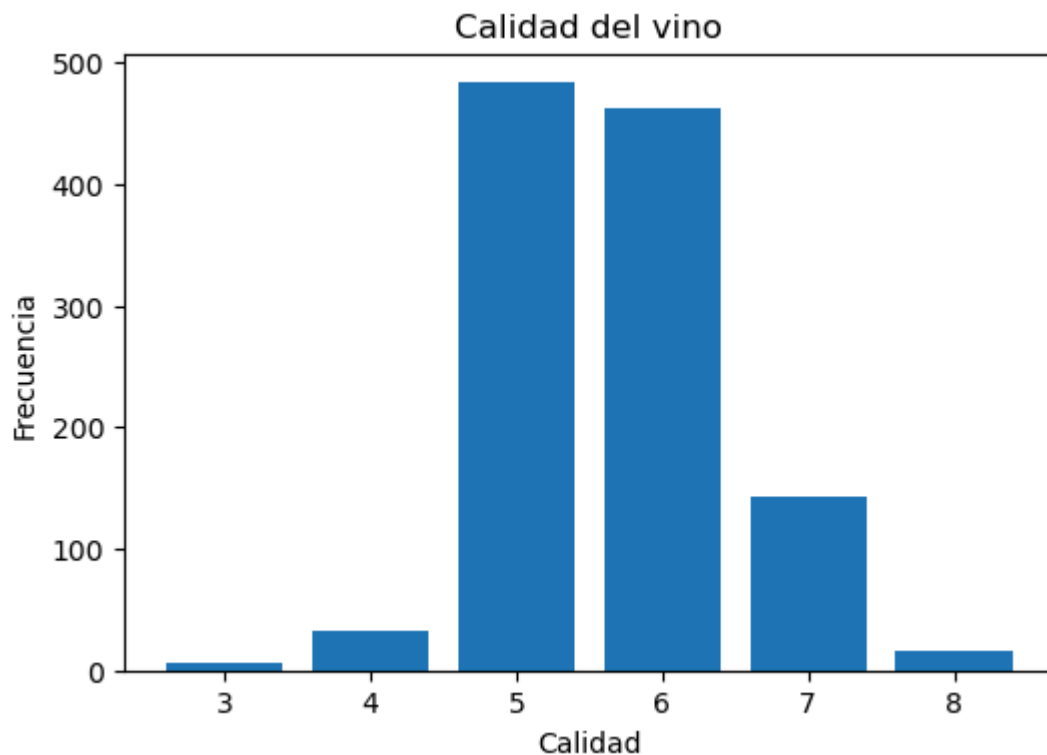
Out[18]:

	count	mean	std	min	25%	50%	75%
fixed acidity	1143.0	8.311111	1.747595	4.60000	7.10000	7.90000	9.100000
volatile acidity	1143.0	0.531339	0.179633	0.12000	0.39250	0.52000	0.640000
citric acid	1143.0	0.268364	0.196686	0.00000	0.09000	0.25000	0.420000
residual sugar	1143.0	2.532152	1.355917	0.90000	1.90000	2.20000	2.600000
chlorides	1143.0	0.086933	0.047267	0.01200	0.07000	0.07900	0.090000
free sulfur dioxide	1143.0	15.615486	10.250486	1.00000	7.00000	13.00000	21.000000
total sulfur dioxide	1143.0	45.914698	32.782130	6.00000	21.00000	37.00000	61.000000
density	1143.0	0.996730	0.001925	0.99007	0.99557	0.99668	0.997845
pH	1143.0	3.311015	0.156664	2.74000	3.20500	3.31000	3.400000
sulphates	1143.0	0.657708	0.170399	0.33000	0.55000	0.62000	0.730000
alcohol	1143.0	10.442111	1.082196	8.40000	9.50000	10.20000	11.100000
quality	1143.0	5.657043	0.805824	3.00000	5.00000	6.00000	6.000000
Id	1143.0	804.969379	463.997116	0.00000	411.00000	794.00000	1209.500000

Análisis de la variable calidad del vino

```
In [20]: calidad_counts = calidad_vino['quality'].value_counts()

plt.figure(figsize=(6, 4))
plt.bar(calidad_counts.index, calidad_counts)
plt.title('Calidad del vino')
plt.xlabel('Calidad')
plt.ylabel('Frecuencia')
plt.show()
```



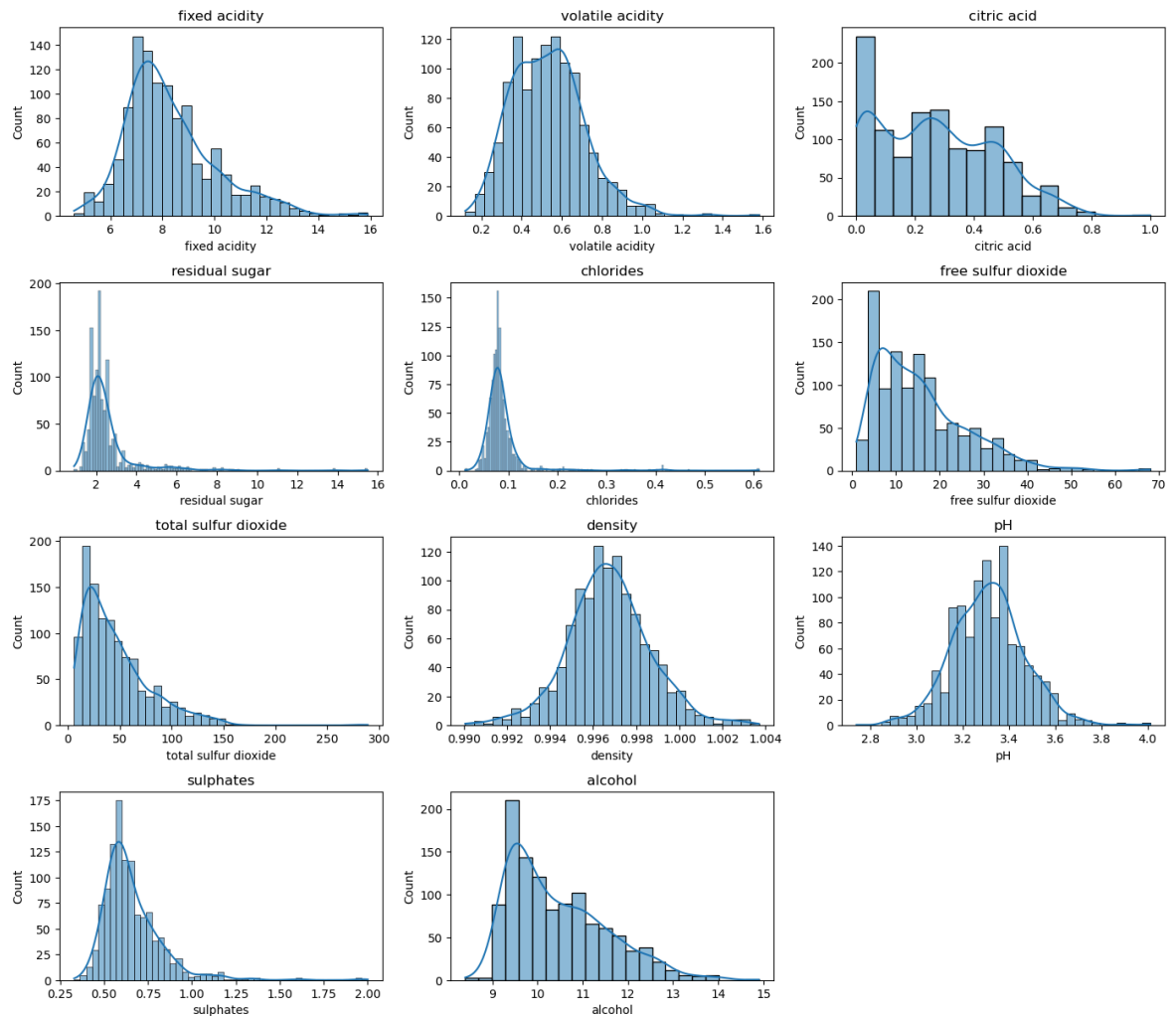
Histogramas junto a la distribución

```
In [23]: columnas_numericas = calidad_vino.select_dtypes(include=["float64"]).columns

n = len(columnas_numericas)

plt.figure(figsize=(14, 3*n))
for idx, feature in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    sns.histplot(calidad_vino[feature], kde=True)
    plt.title(feature)

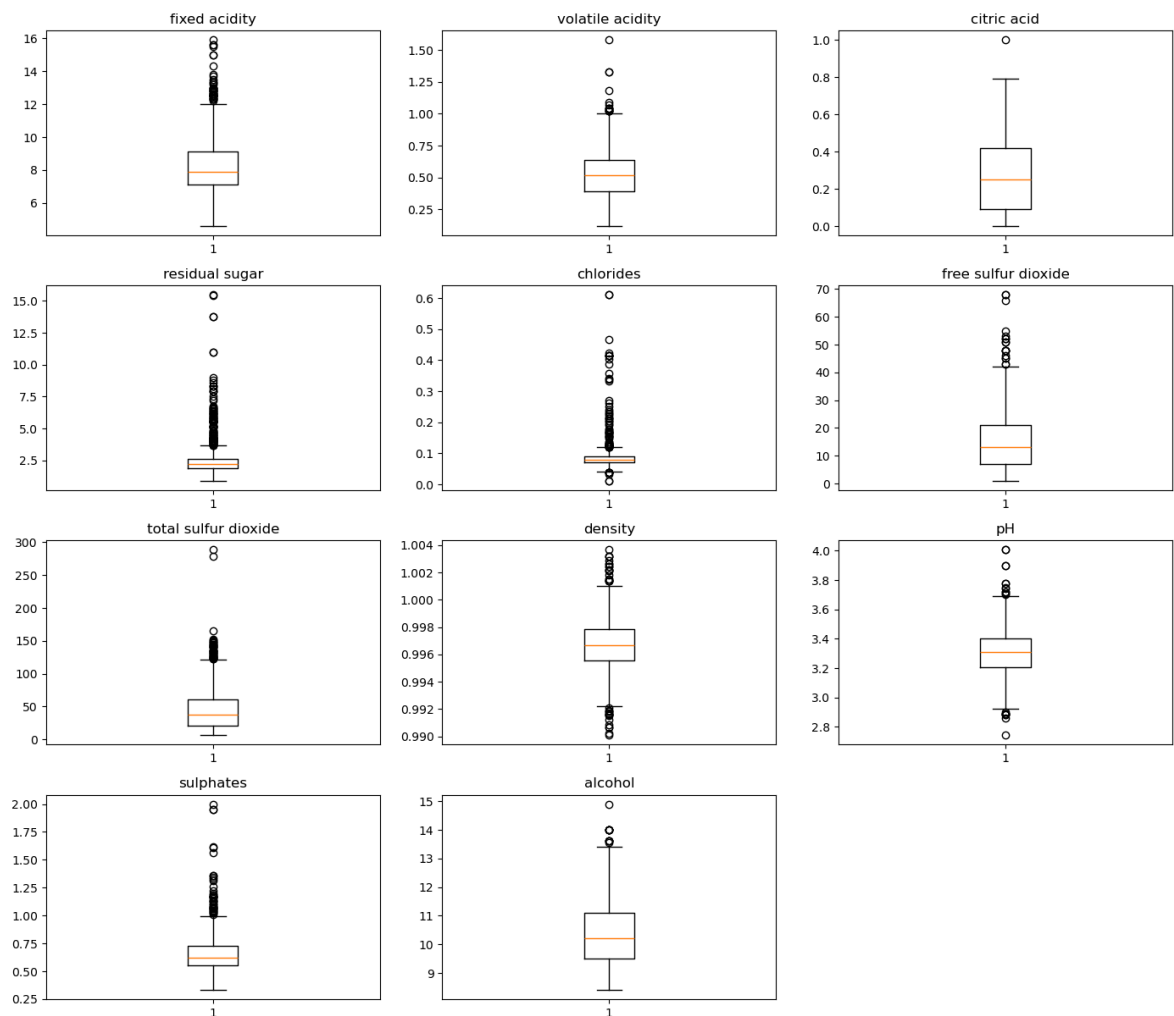
plt.tight_layout()
plt.show()
```



Visualización de valores atípicos

```
In [25]: plt.figure(figsize=(14, 3*n))
for idx, feature in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    plt.boxplot(calidad_vino[feature])
    plt.title(feature)

plt.tight_layout()
plt.show()
```



Métodos para detectar valores atípicos

```
In [27]: #Método IQR
def outliers_IQR(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

#Método de la desviación estándar
def outliers_desv(x):
    mu = x.mean()
    sigma = x.std()
    li = mu - 3 * sigma
    ls = mu + 3 * sigma
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

#Método de la puntuación z
def outliers_z(df, n, features):
    outlier_list = []

    for column in features:
        data_media = df[column].mean()
```

```

        data_std = df[column].std()
        umbral = 3
        z_score = abs( (df[column] - data_media)/data_std )
        outlier_list_column = df[z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[z_score > umbral]

    return df1.shape[0]

from scipy.stats import median_abs_deviation
#Método de la puntuación z modificado
def z_scoremod_method(df,n,var):
    outlier_list = []

    for column in var:

        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        MAD = median_abs_deviation

        mod_z_score = abs(0.6745*(df[column] - data_media)/MAD(df[column]))
        outlier_list_column = df[mod_z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[mod_z_score > umbral]

    return df1.shape[0]

```

```

In [29]: #Aplicar método IQR
for vars in columnas_numericas:
    num_outliers = outliers_IQR(calidad_vino[vars])
    print("El número de outliers de la variable %s es %s" %(vars, num_outliers))

```

```

El número de outliers de la variable fixed acidity es 44
El número de outliers de la variable volatile acidity es 14
El número de outliers de la variable citric acid es 1
El número de outliers de la variable residual sugar es 110
El número de outliers de la variable chlorides es 77
El número de outliers de la variable free sulfur dioxide es 18
El número de outliers de la variable total sulfur dioxide es 40
El número de outliers de la variable density es 36
El número de outliers de la variable pH es 20
El número de outliers de la variable sulphates es 43
El número de outliers de la variable alcohol es 12

```

```

In [31]: #Aplicar todos Los métodos
lista_IQR = [outliers_IQR(calidad_vino[vars]) for vars in columnas_numericas]
lista_desv = [outliers_desv(calidad_vino[vars]) for vars in columnas_numericas]
lista_z = [outliers_z(calidad_vino,1,[vars]) for vars in columnas_numericas]
lista_modz = [z_scoremod_method(calidad_vino,1,[vars]) for vars in columnas_nume

total_outliers = pd.DataFrame({"Variables": columnas_numericas, "Método IQR": li

```

```
"Método desviación": lista_desv, "Método puntuación z modificado": lista_modz})
```

```
In [33]: total_outliers
```

```
Out[33]:
```

	Variables	Método IQR	Método desviación	Método puntuación Z	Método puntuación z modificado
0	fixed acidity	44	9	9	39
1	volatile acidity	14	5	5	6
2	citric acid	1	1	1	0
3	residual sugar	110	23	23	104
4	chlorides	77	21	21	65
5	free sulfur dioxide	18	13	13	18
6	total sulfur dioxide	40	10	10	33
7	density	36	12	12	24
8	pH	20	5	5	8
9	sulphates	43	21	21	42
10	alcohol	12	7	7	12

El método IQR identifica la mayor cantidad de observaciones atípicas en la mayoría de las variables, especialmente en residual sugar, con 110 casos, seguido de chlorides, con 77, fixed acidity, con 44, y sulphates, con 43. Los métodos basados en la desviación estándar y en la puntuación Z producen resultados idénticos y más restrictivos, ya que detectan una cantidad considerablemente menor de valores atípicos; por ejemplo, solo 23 casos en residual sugar y 21 en chlorides y sulphates. Por su parte, la puntuación Z modificada presenta resultados cercanos al método IQR en variables como residual sugar, chlorides, sulphates y fixed acidity, lo que puede indicar distribuciones asimétricas o con valores extremos. La variable citric acid presenta únicamente un valor atípico con los primeros tres métodos y ninguno con la puntuación Z modificada, mientras que alcohol muestra pocos casos en todos los procedimientos

Eliminar outliers de Residual sugar

```
In [36]: def elimina_outliers(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    indices_sin_outliers = (x>=li) & (x<=ls)
    sin_outliers = x[indices_sin_outliers]
    return sin_outliers
```

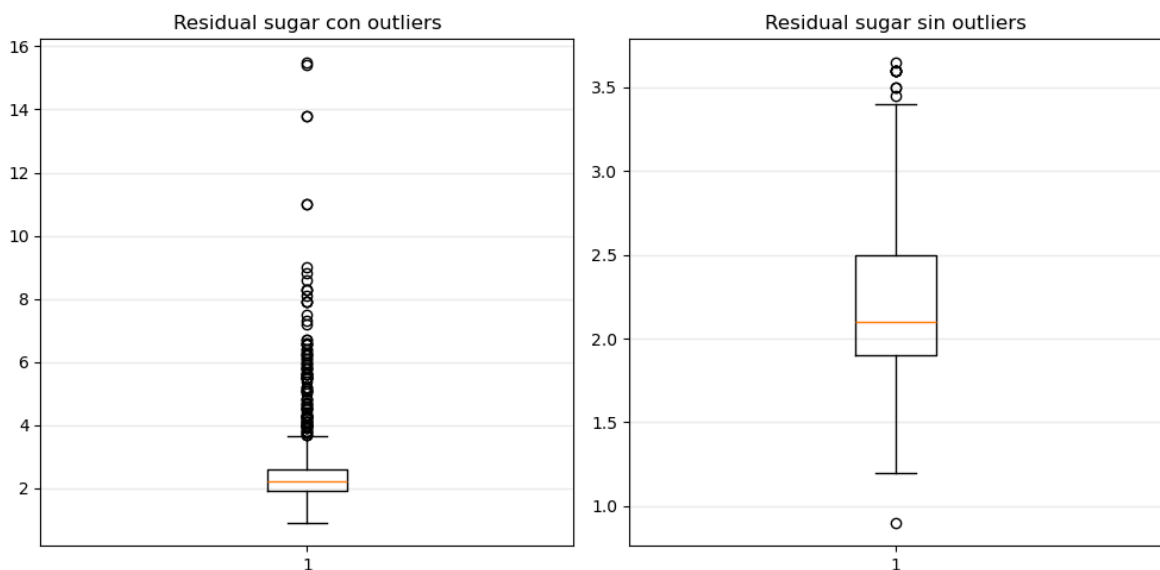
```
In [38]: residual_sugar_sin_outliers = elimina_outliers(calidad_vino["residual sugar"])
```

```
In [40]: fig, ax = plt.subplots(1, 2, figsize=(10, 5))

ax[0].boxplot(calidad_vino["residual sugar"])
ax[0].set_title("Residual sugar con outliers")
ax[0].set_xticks([1])
ax[0].grid(axis="y", alpha=0.2)

ax[1].boxplot(residual_sugar_sin_outliers)
ax[1].set_title("Residual sugar sin outliers")
ax[1].set_xticks([1])
ax[1].grid(axis="y", alpha=0.2)

plt.tight_layout()
plt.show()
```



Análisis de valores anómalos

```
In [42]: from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=2026)

calidad_vino["Anomalías"] = iso.fit_predict(calidad_vino[columnas_numericas])
calidad_vino['Puntaje de anomalía'] = iso.decision_function(calidad_vino[columna
```

```
In [43]: calidad_vino
```


Out[43]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulph
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	
...	...	...	...	...	...	...	...	...	...	...
1138	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	
1139	6.8	0.620	0.08	1.9	0.068	28.0	38.0	0.99651	3.42	
1140	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	
1141	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	
1142	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	

1143 rows × 15 columns

In [44]: `calidad_vino['Anomalias'].value_counts()`

Out[44]: Anomalias
 1 1086
 -1 57
 Name: count, dtype: int64

In [45]: `plt.figure()
 calidad_vino['Puntaje de anomalía'].hist(bins=15,edgecolor="black")
 plt.show()`

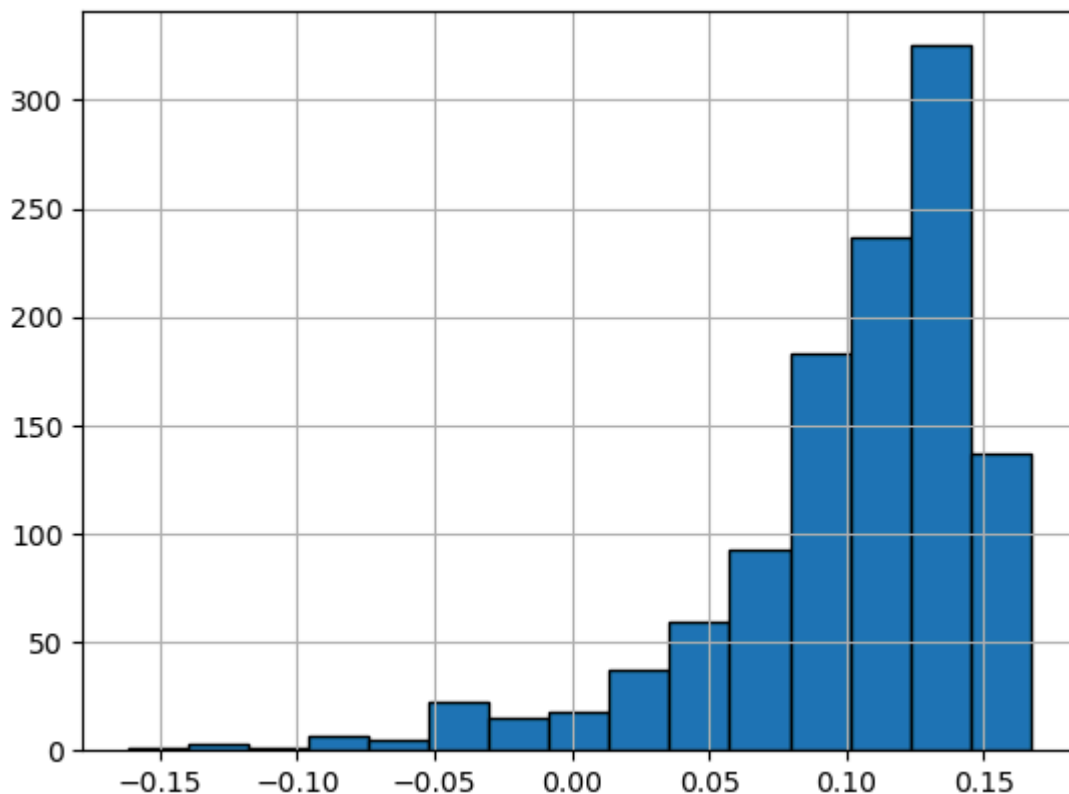
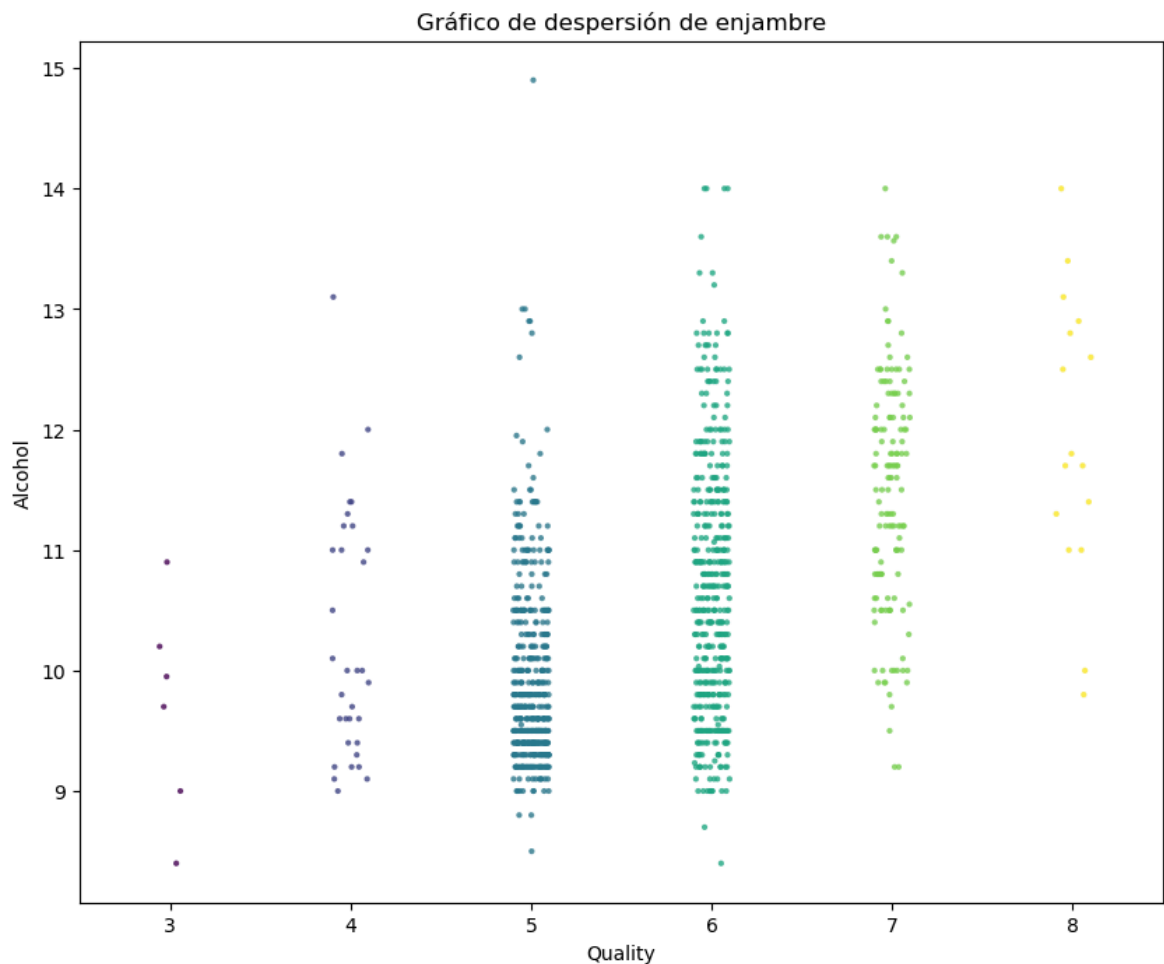


Gráfico de dispersión de enjambre

El gráfico de dispersión de enjambre, también llamado swarm plot, representa cada observación mediante un punto y permite comparar la distribución de una variable numérica entre una o varias categorías. Los puntos se colocan alrededor del eje correspondiente a cada categoría, pero se desplazan ligeramente hacia los lados para evitar que se superpongan; por eso visualmente parecen formar un "enjambre". A diferencia del diagrama de caja, este gráfico permite observar directamente todos los valores, las zonas donde se concentran, la dispersión, posibles grupos y valores atípicos.

```
In [59]: plt.figure(figsize=(10, 8))

#sns.swarmplot(x="quality", y="alcohol", data=calidad_vino, palette='viridis')
sns.stripplot(x="quality", y="alcohol", hue="quality", data=calidad_vino, palette=
    jitter=True, alpha=0.8, size=3)
plt.title("Gráfico de dispersión de enjambre")
plt.xlabel('Quality')
plt.ylabel('Alcohol')
plt.show()
```



El diagrama de Violín

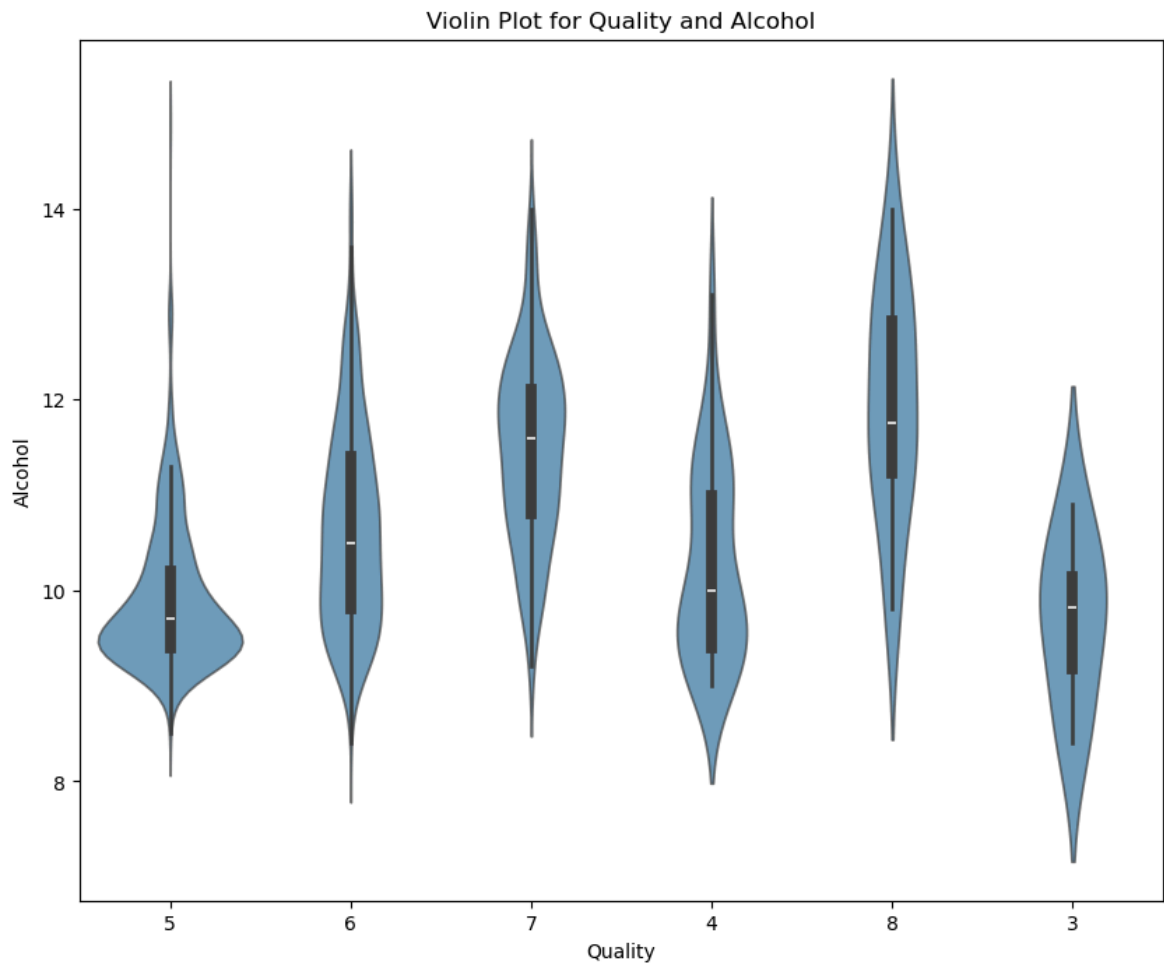
El diagrama de violín es una gráfica que muestra la distribución de una variable numérica en una o varias categorías. Combina información de un diagrama de caja con una curva de densidad: la parte ancha del violín indica que hay muchas observaciones concentradas en ese intervalo, mientras que las partes estrechas indican menor frecuencia

```
In [66]: calidad_vino['quality'] = calidad_vino['quality'].astype(str)

plt.figure(figsize=(10, 8))

sns.violinplot(x="quality", y="alcohol", data=calidad_vino, alpha=0.7)

plt.title('Violin Plot for Quality and Alcohol')
plt.xlabel('Quality')
plt.ylabel('Alcohol')
plt.show()
```



Análisis de correlaciones

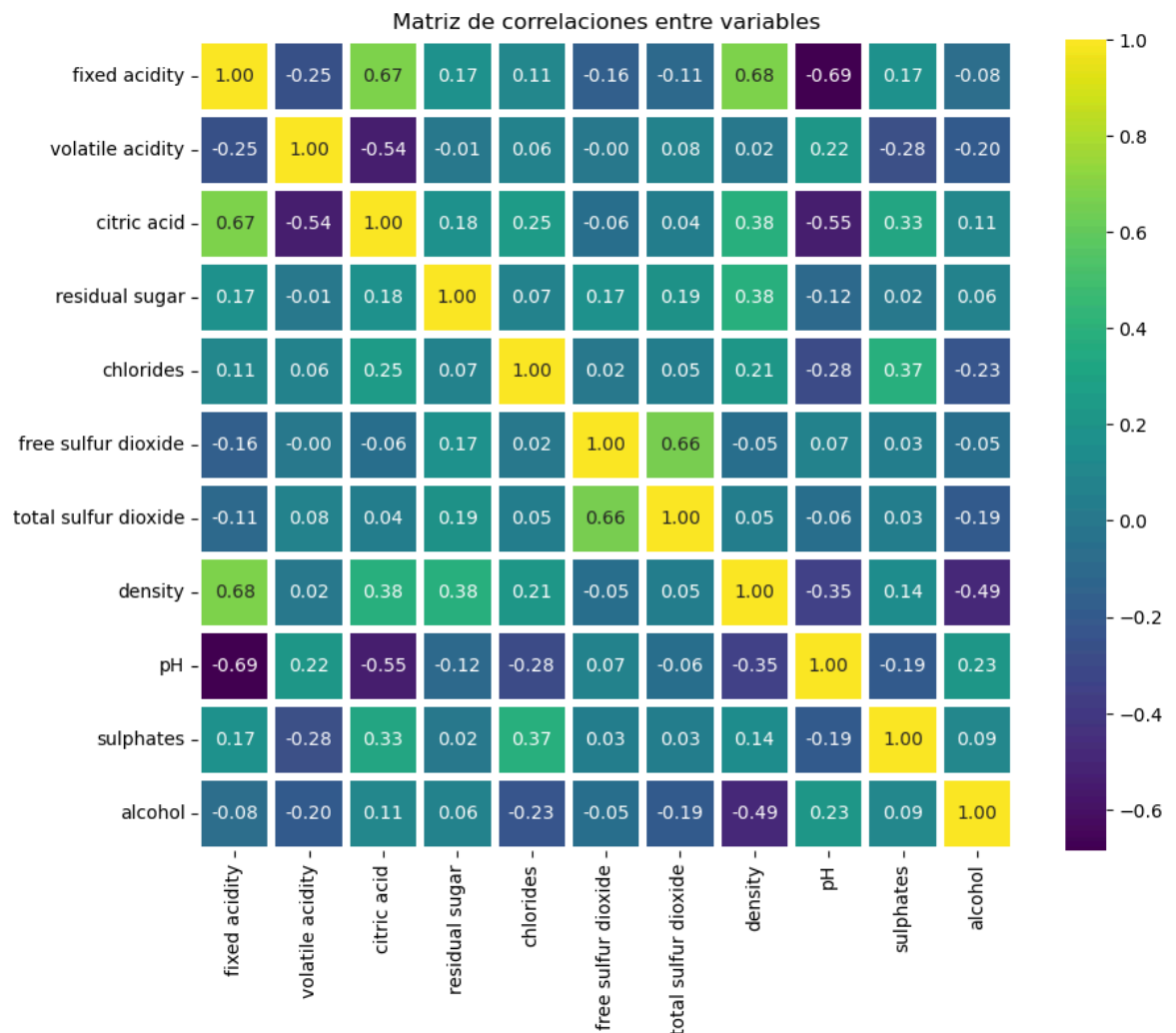
```
In [67]: import seaborn as sns

# Calculamos la matriz de coeficientes de correlación entre las variables numéri
correlacion = calidad_vino[columnas_numericas].corr()

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion, annot=True, fmt='.2f', linewidths=3, cmap='viridis')

plt.title('Matriz de correlaciones entre variables')
plt.show()
```



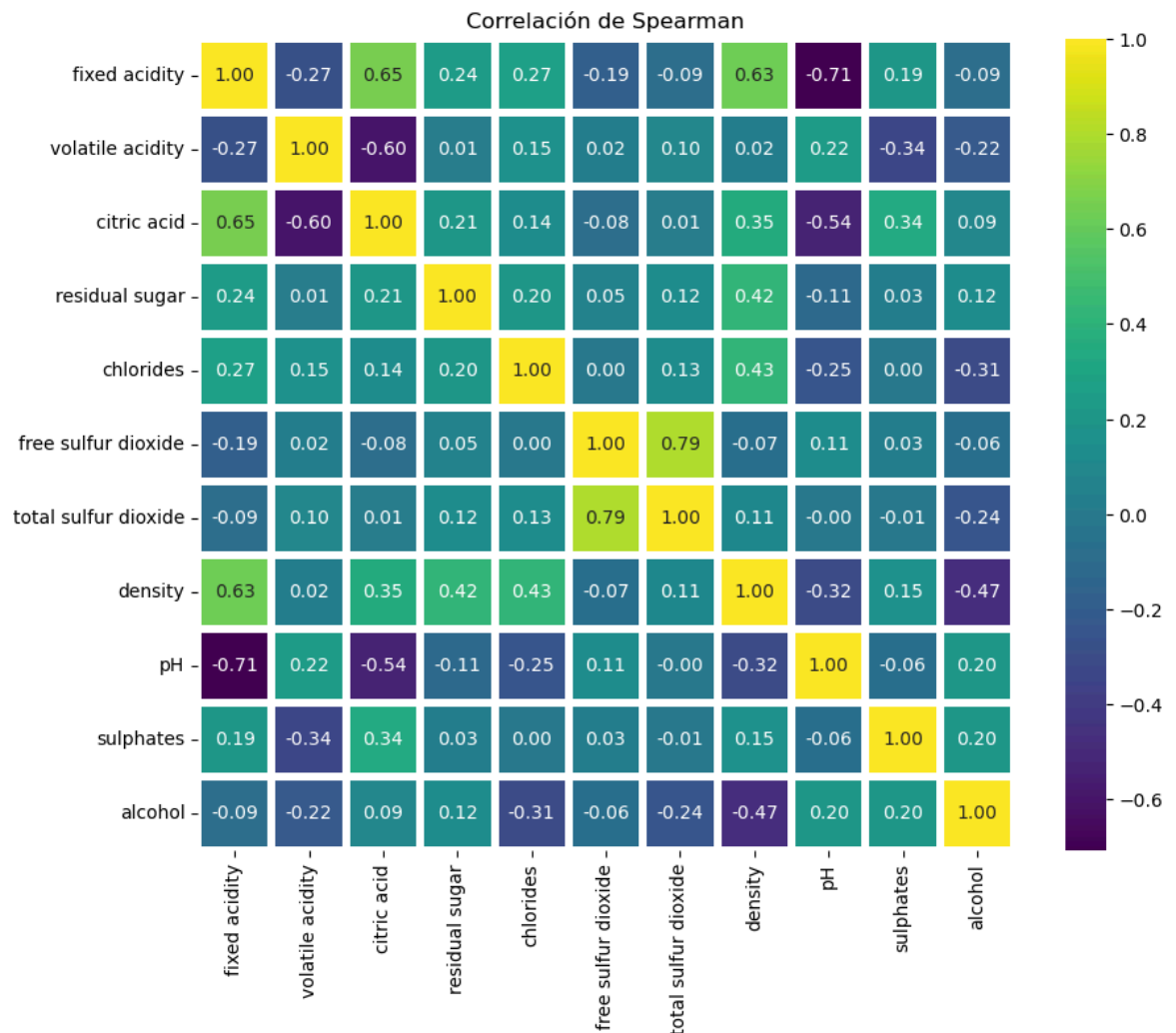
La matriz de correlación muestra que las relaciones lineales más fuertes se presentan entre fixed acidity y pH, con una correlación negativa de -0.69 , lo que indica que al aumentar la acidez fija el pH tiende a disminuir, y entre fixed acidity y density, con una correlación positiva de 0.68 . También destacan las correlaciones positivas entre fixed acidity y citric acid de 0.67 , y entre free sulfur dioxide y total sulfur dioxide es de 0.66 . Por otra parte, citric acid presenta relaciones negativas moderadas con pH con -0.55 y volatile acidity de -0.54 . El contenido de alcohol se relaciona negativamente con density con -0.49 , lo que sugiere que los vinos con mayor grado alcohólico tienden a tener menor densidad.

```
In [72]: # Calculamos la matriz de coeficientes de correlación entre las variables numéricas
correlacion_spearman = calidad_vino[columnas_numericas].corr(method='spearman')

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion_spearman, annot=True, fmt='.2f', linewidths=3, cmap='viridis')

plt.title('Correlación de Spearman')
plt.show()
```



La matriz de correlación de Spearman muestra que las asociaciones monotónicas más fuertes se presentan entre free sulfur dioxide y total sulfur dioxide, con un coeficiente de 0.79, lo que indica que ambas variables tienden a aumentar conjuntamente, y entre fixed acidity y pH, con una correlación negativa de -0.71 , es decir, a mayor acidez fija, menor pH.

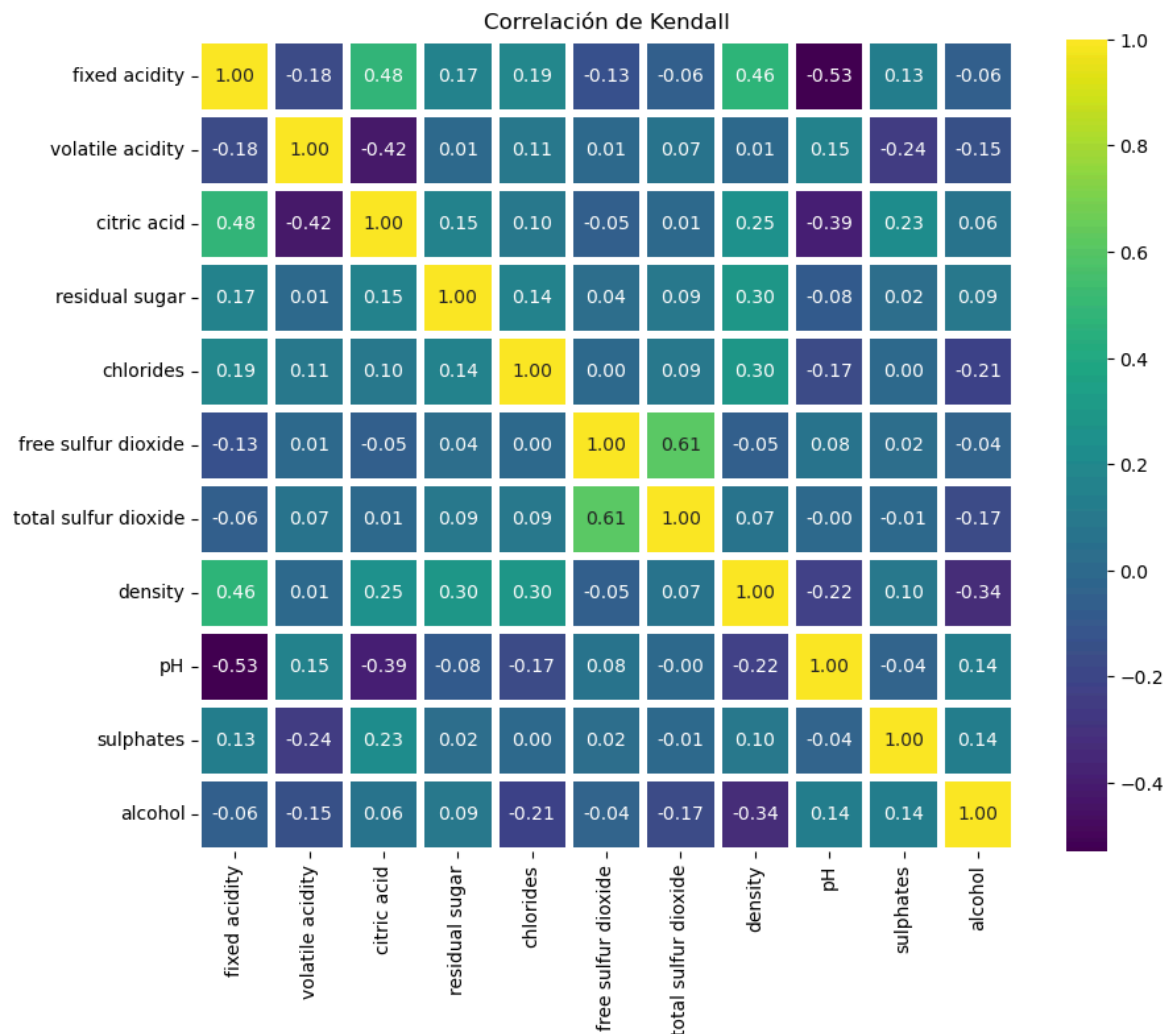
```
In [77]: import seaborn as sns

# Calculamos la matriz de coeficientes de correlación entre las variables numéricas
correlacion_kendall = calidad_vino[columnas_numericas].corr(method="kendall")

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion_kendall, annot=True, fmt='.2f', linewidths=3, cmap='viri

plt.title('Correlación de Kendall')
plt.show()
```



La matriz de correlación de Kendall muestra que la asociación monotónica más fuerte se presenta entre free sulfur dioxide y total sulfur dioxide, con un coeficiente de 0.61, lo que indica que ambas variables tienden a aumentar conjuntamente

Implementación de KNN para analizar el efecto sobre la calidad del vino

```
In [83]: ##Implementar el algoritmo KNN
X_independientes = calidad_vino[columnas_numericas]
y_dependiente = calidad_vino["quality"]

#Estandarizar variables independientes

from sklearn.preprocessing import StandardScaler

escalar = StandardScaler()
escalar.fit(X_independientes)
X_independientes_st = escalar.transform(X_independientes)
```

```
In [117... from sklearn.neighbors import KNeighborsClassifier

model_KNN_5 = KNeighborsClassifier(n_neighbors =5)
model_KNN_5.fit(X_independientes, y_dependiente)
```

Out[117...

▼ KNeighborsClassifier ⓘ ?

KNeighborsClassifier()

In [119...

```
y_pred = model_KNN_5.predict(X_independientes_st)
```

C:\Users\USER\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names

```
warnings.warn(
```

In [121...

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_dependiente, y_pred)
print("El accuracy score que se tiene es %s" %accuracy)
```

El accuracy score que se tiene es 0.14610673665791776

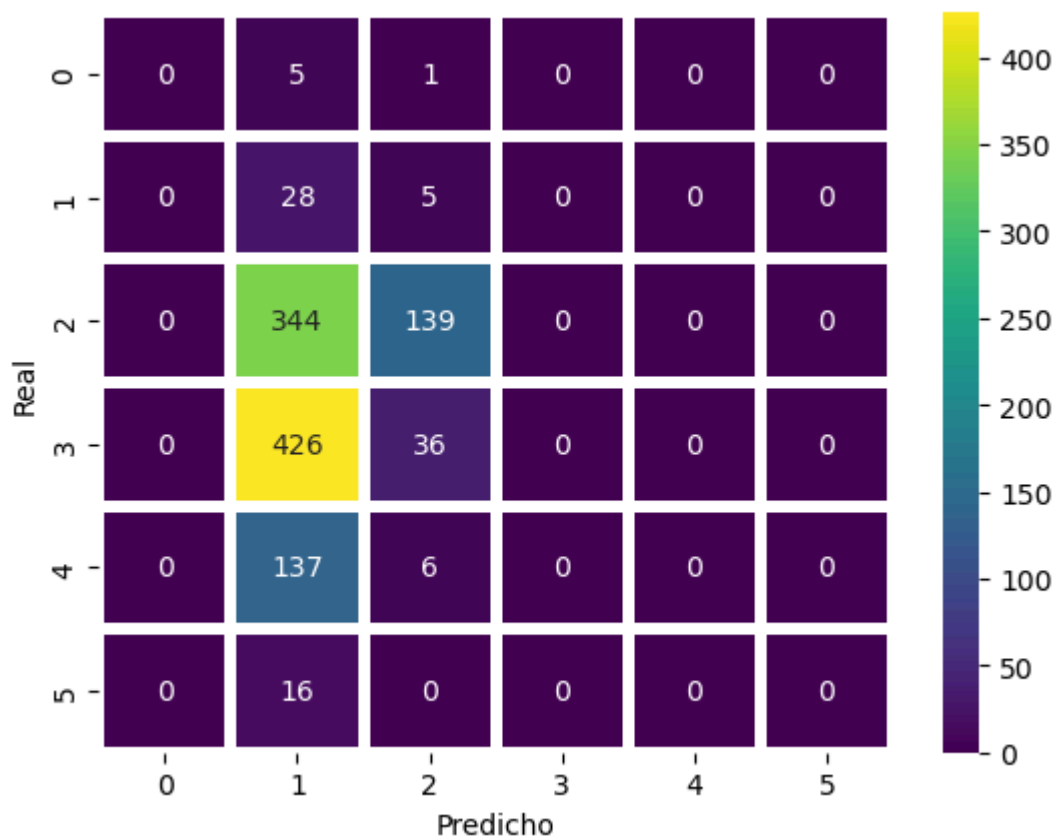
In [123...

```
from sklearn.metrics import confusion_matrix

#Obtener la matriz de confusión
cm = confusion_matrix(y_dependiente, y_pred)
sns.heatmap(cm, annot=True, fmt='d', linewidths=3, cmap='viridis')
plt.xlabel("Predicho")
plt.ylabel("Real")
```

Out[123...

Text(50.72222222222214, 0.5, 'Real')



In [103...

```
# Encontrar el valor optimo de K
from sklearn.model_selection import cross_val_score
#Valores de K que vamos a probar

k_range = range(1, 100)
#Inicializar vector de puntuaciones
```



```

k_scores = []
# bucle
for k in k_range:
    # ajustar el modelo con k vecinos
    knn = KNeighborsClassifier(n_neighbors=k)
    # obtener puntuaciones de VC
    scores = cross_val_score(knn,X_independientes,y_dependiente ,cv = 5, scoring
    # Guardar puntuaciones en el vector
    k_scores.append(scores.mean())

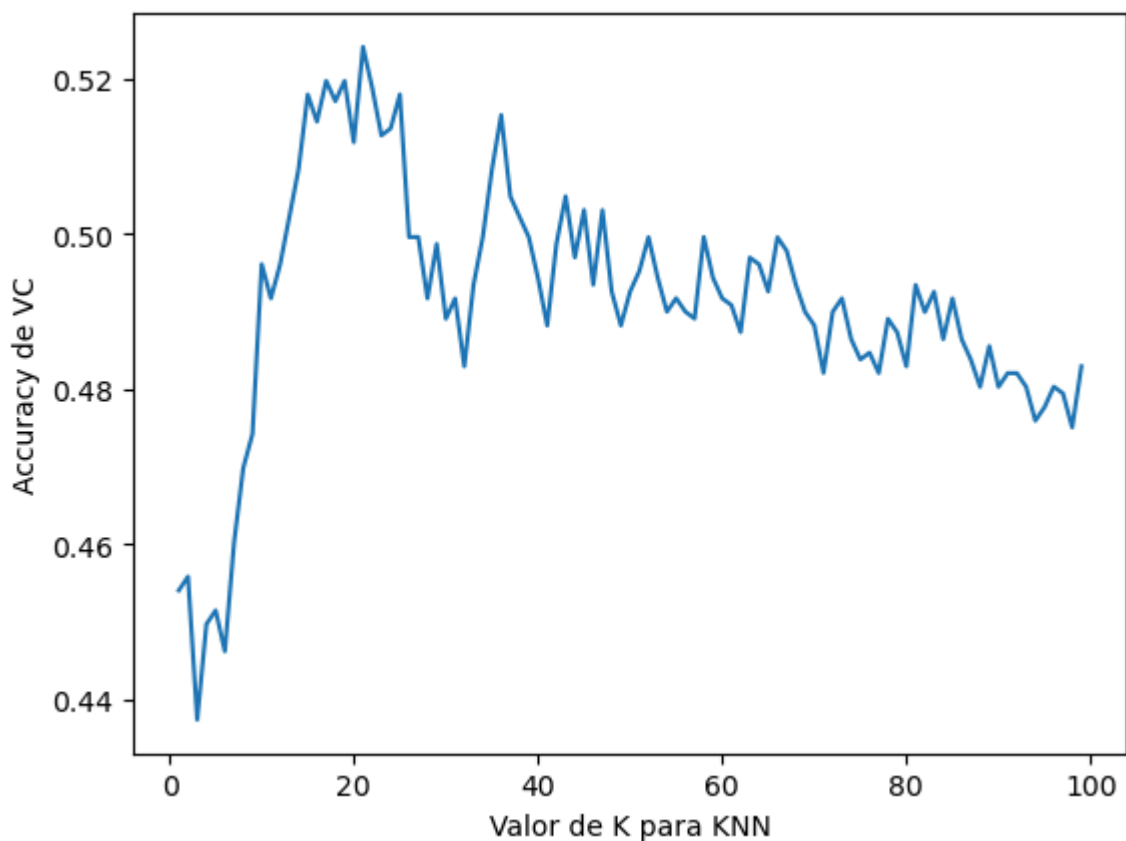
```

In [106... *#Representar la validación cruzada*

```

plt.plot(k_range, k_scores)
plt.xlabel('Valor de K para KNN')
plt.ylabel('Accuracy de VC')
plt.show()

```



In [109... `print("El valor de K que maximiza la accuracy es", np.argmax(k_scores) + 1)`

El valor de K que maximiza la accuracy es 21

In []: